

DSC630: Predicting Homelessness Rates From Continuum of Care Features

Jess South

November 22, 2025

1 DSC630: Predicting Homelessness Rates From Continuum of Care Features

1.1 Data import and preparation

```
[1]: # Imports
import pandas as pd
```

```
[2]: # Import data
src = './05b_analysis_file_update.csv'
df = pd.read_csv(src)
df.head()
```

```
[2]:
```

	year	cocnumber	pit_tot_shelt_pit_hud	pit_tot_unshelt_pit_hud	\
0	2010	AK-500	1113.0	118.0	
1	2011	AK-500	1082.0	141.0	
2	2012	AK-500	1097.0	50.0	
3	2013	AK-500	1070.0	52.0	
4	2014	AK-500	970.0	53.0	

	pit_tot_hless_pit_hud	pit_ind_shelt_pit_hud	pit_ind_unshelt_pit_hud	\
0	1231.0	633.0	107.0	
1	1223.0	677.0	117.0	
2	1147.0	756.0	35.0	
3	1122.0	792.0	52.0	
4	1023.0	688.0	48.0	

	pit_ind_hless_pit_hud	pit_perfam_shelt_pit_hud	\
0	740.0	480.0	
1	794.0	405.0	
2	791.0	341.0	
3	844.0	278.0	
4	736.0	282.0	

	pit_perfam_unshelt_pit_hud	...	sub_high_cost_rent75	\
0	11.0	...	1	

1	24.0	...	1
2	15.0	...	1
3	0.0	...	1
4	5.0	...	1

	sub_high_cost_homeval75	sub_high_rent_share75	tight_high_cost_rental_mkt	\
0	1	1	3	
1	1	0	3	
2	1	1	3	
3	1	0	3	
4	1	1	3	

	sub_tight_high_cost_rent	sub_west_coast_all_urb	sub_west_census	\
0	1	1	1	
1	1	1	1	
2	1	1	1	
3	1	1	1	
4	1	1	1	

	major_city	suburban	rural
0	1	0	0
1	1	0	0
2	1	0	0
3	1	0	0
4	1	0	0

[5 rows x 332 columns]

```
[3]: # Get shape of df
df.shape
```

[3]: (3008, 332)

1.1.1 Remove final unwanted variables

These variables contribute no value to a predictive model, being either outcome-focused, time-related, or location-focused:

```
[4]: # Remove nonnumerical unwanted variables
df = df.drop(['year', 'cocnumber', 'state_abr'], axis=1)
```

```
[5]: # Remove other outcome variables after deriving target
drop_cols = [
    'pit_tot_shelt_pit_hud',
    'pit_tot_unshelt_pit_hud',
    'pit_tot_hless_pit_hud',
    'pit_ind_shelt_pit_hud',
    'pit_ind_unshelt_pit_hud',
```

```

'pit_ind_hless_pit_hud',
'pit_perfam_shelt_pit_hud',
'pit_perfam_unshelt_pit_hud',
'pit_perfam_hless_pit_hud',
'pit_ind_chronic_hless_pit_hud',
'pit_perfam_chronic_hless_pit_hud',
'pit_vet_hless_pit_hud',
'econ_urb_urbanicity',
'coctag',
'panelvar',
'hous_pol_totalind_hud',
'hous_pol_totalday_hud',
'hous_pol_totalexit_hud',
'hous_pol_numret6mos_hud',
'hous_pol_numret12mos_hud',
'hous_pol_fedfundcoc',
'hous_pol_fund_project',
'hous_pol_bed_es_hic_hud',
'hous_pol_bed_oph_hic_hud',
'hous_pol_bed_psh_hic_hud',
'hous_pol_bed_rrh_hic_hud',
'hous_pol_bed_sh_hic_hud',
'hous_pol_bed_th_hic_hud',
'hous_pol_perm_bed_hic_hud',
'hous_pol_temp_bed_hic_hud',
'pit_miss',
'dem_pop_density_census',
'pit_shelt_pit_hud_share',
'pit_unshelt_pit_hud_share',
'census_division',
'census_region',
'state'
]
df = df.drop(drop_cols, axis=1)

```

These variables will be removed due to being year-specific (information better represented in other aggregate fields):

```

[6]: # Remove yearly outcome variables
import re
drop_cols = [c for c in df.columns if re.search(r'\d{4}$', str(c))]
df = df.drop(drop_cols, axis=1)

```

1.2 Feature selection

```
[7]: # Imports
from sklearn.model_selection import train_test_split
from sklearn.feature_selection import VarianceThreshold
from sklearn.impute import SimpleImputer
```

```
[8]: # Impute NaN values for variance threshold
# Drop columns with too many NaNs
nan_mean = df.isna().mean()
drop_cols = nan_mean[nan_mean > 0.5].index.tolist()
df = df.drop(columns=drop_cols)
print(f"Dropped {len(drop_cols)} columns.")
```

Dropped 81 columns.

```
[9]: # Get logarithmic pit_hless_pit_hud_share to account for skew
import numpy as np
df['log_hless'] = np.log(df['pit_hless_pit_hud_share'])
```

```
[10]: # Remove other outcome variables after deriving target
df_old = df
drop_cols = [
    'pit_hless_pit_hud_share'
]
df = df.drop(drop_cols, axis=1)
```

```
[11]: # Drop rows where target is NaN
target = 'log_hless' # Rate of total homeless per 10k people
df = df.dropna(subset=[target])
df.shape
```

```
[11]: (2994, 151)
```

```
[12]: # Drop outliers where target var is more than 1.5x the mean
mean_target = df[target].mean()
df = df[df[target] <= (mean_target * 1.5)]
df.shape
```

```
[12]: (2847, 151)
```

```
[13]: # Train/test split
X = df.drop(target, axis=1)
y = df[target]
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=0,
    shuffle=True # Ignoring time/year for this model
```

```
)
```

```
[14]: # Impute remaining NaNs
imputer = SimpleImputer(strategy='median')
imputer.fit(X_train)
X_train = pd.DataFrame(imputer.transform(X_train), columns=X_train.columns,
↳ index=X_train.index)
X_test = pd.DataFrame(imputer.transform(X_test), columns=X_test.columns,
↳ index=X_test.index)
```

```
[15]: # Get shape of X_train for later comparison
X_train.shape
```

```
[15]: (2277, 150)
```

```
[16]: # Variance threshold >=80% variance
vt = VarianceThreshold(threshold=(0.8 *(1 - 0.8)))
vt.fit(X_train)
```

```
[16]: VarianceThreshold(threshold=0.15999999999999998)
```

```
[17]: # Get features
mask = vt.get_support()
selected_cols = X_train.columns[mask]
X_train = X_train[selected_cols]
X_test = X_test[selected_cols]
```

```
[18]: # Get shape of X_train for comparison
X_train.shape
```

```
[18]: (2277, 122)
```

Using a variance threshold of >80%, the number of columns has been reduced from 329 to 211.

1.3 Select best features using KMeans

```
[19]: # Imports
from sklearn.feature_selection import SelectKBest, f_regression
```

```
[20]: # Create selector
k = 20
selector = SelectKBest(score_func=f_regression, k=k)
selector.fit(X_train, y_train)
```

```
[20]: SelectKBest(k=20, score_func=<function f_regression at 0x788f29f60670>)
```

```
[21]: # Get selected features
mask = selector.get_support()
```

```
selected_cols = X_train.columns[mask].tolist()
ranks = pd.Series(selector.scores_, index=X_train.columns).nlargest(k)
ranks
```

```
[21]: urban_cat          413.612862
      major_city        333.394332
      hou_mkt_rentshare_xt  222.449011
      rural             220.972212
      hou_mkt_burden_sev_own_acs_diff  215.192569
      dem_soc_white_census_share  210.551017
      hou_pol_hlessconduct_sleep  206.152987
      hou_pol_hlessconduct_total  198.843791
      hou_mkt_burden_own_xt  151.993803
      hou_mkt_burden_rent_xt  150.695323
      hou_mkt_burden_sev_own_xt  143.397119
      econ_labor_midskill_xt  137.873504
      dem_soc_hispanic_census_share  131.645467
      hou_pol_eli_psh_hud  131.158833
      dem_soc_asian_census_share  123.250287
      hou_mkt_homeage_acs5yr_diff  122.062723
      env_wea_avgtemp_noaa  120.516303
      hou_mkt_burden_own_acs5yr_diff  116.431063
      hou_pol_hudunit_psh_hud_share  114.686280
      dem_soc_pacific_census  113.309335
      dtype: float64
```

From these best 20 features, I've selected:

- `hou_mkt_burden_sev_own_acs_diff`: The change in the percentage of home owners with severe cost burden (between 2016 and 2011). This is a derived value that nevertheless reflects a region's shift in cost burden, either increasing or decreasing.
- `env_wea_avgtemp_noaa`, the average January temperature.
- `hou_mkt_density_census`, housing units estimate divided by square miles (representing housing availability).
- `hou_mkt_rentshare_xt`, the share of renter-occupied housing units to all occupied housing units (representing the ratio of renters to owners, which in turn reflects a local housing market).
- `hou_mkt_burden_sev_own_xt`, the share of owned housing units with 50+% monthly owner costs to income.
- `hou_mkt_burden_rent_xt` and `hou_mkt_burden_own_xt`, the share of rented/owned housing units with 30+% monthly owner costs to income.
- `hou_mkt_homeval_xt`, log of median housing unit value.
- `econ_sn_ssi_part_rate_SSA`, the SSI (Supplemental Security Income) participation rate (reflecting the proportion of a population that has disabilities, is older, or has low income).
- `hou_mkt_ovrcrowd_xt`, the share of occupied housing units with occupants per room exceeding 1.5.

I am omitting: - `dem_soc_white_census_share`, reflecting the number of white people in a population. This is interesting information but not applicable to a predictive model as the users of this

model cannot (and should not) change the population makeup to affect outcomes. For the same reason, I am omitting `dem_soc_asian_census_share` and `dem_soc_hispanic_census_share`. - `hou_pol_hlessconduct_sleep` and `hou_pol_hlessconduct_total`, the former being the count of sleeping, camping, lying/sitting, and vehicle restriction laws, and the latter being the count of all homeless conduct laws. These are laws intended not to affect the number of homeless people but to police their behavior (and therefore possibly push them into other areas). Because moving unhoused persons into another region does not actually reduce the number of unhoused persons, this is nonapplicable. - `dem_pop_adult_census_share` and `dem_pop_child_census_share`, as these are likely just to reflect population count.

[22]: *# Limit datasets to the selected nine features*

```
features = [
    'urban_cat',
    'rural',
    'hou_mkt_burden_sev_own_acs_diff',
    'env_wea_avgtemp_noaa',
    'hou_mkt_density_census',
    'hou_mkt_burden_sev_own_xt',
    'hou_mkt_rentshare_xt',
    'hou_mkt_burden_own_xt',
    'hou_mkt_burden_rent_xt',
    'hou_mkt_homeval_xt',
    'econ_sn_ssi_part_rate_SSA',
    'hou_mkt_ovrcrowd_xt',
    'econ_labor_midskill_xt',
    'hou_pol_eli_psh_hud',
    'hou_mkt_homeage_acs5yr_diff',
    'hou_pol_hudunit_psh_hud_share'
]
X_train = X_train[features]
X_test = X_test[features]
X_train.head()
```

```
[22]:      urban_cat  rural  hou_mkt_burden_sev_own_acs_diff  env_wea_avgtemp_noaa  \
1551         1.0    1.0                -0.940786                17.752237
1178         2.0    0.0                -0.574791                33.369999
752          3.0    0.0                 0.224956                41.290001
1595         3.0    0.0                 0.360227                30.850969
473          3.0    0.0                -1.683241                28.040001

      hou_mkt_density_census  hou_mkt_burden_sev_own_xt  hou_mkt_rentshare_xt  \
1551                0.107976                6.203594                24.775291
1178                0.785223                9.688565                31.915245
752                2.692289                9.688565                31.915245
1595                0.397571                6.727179                31.485823
473                1.190305               10.638936                34.397709
```

	hou_mkt_burden_own_xt	hou_mkt_burden_rent_xt	hou_mkt_homeval_xt	\
1551	18.103697	41.040764	125.328163	
1178	25.997095	50.275688	168.500000	
752	25.997095	50.275688	168.500000	
1595	19.308041	45.676907	111.041206	
473	29.787689	48.970360	217.000000	

	econ_sn_ssi_part_rate_SSA	hou_mkt_ovrcrowd_xt	econ_labor_midskill_xt	\
1551	1.369456	1.841597	96.945396	
1178	2.960381	1.960686	92.835297	
752	3.715701	1.960686	92.835297	
1595	2.339186	1.229247	93.270851	
473	1.183726	2.069638	91.762337	

	hou_pol_eli_psh_hud	hou_mkt_homeage_acs5yr_diff	\
1551	64.177010	1.945923	
1178	79.000000	1.000000	
752	75.000000	1.000000	
1595	66.997147	0.060181	
473	84.000000	2.000000	

	hou_pol_hudunit_psh_hud_share
1551	3.574198
1178	5.332373
752	8.790756
1595	4.510035
473	1.995623

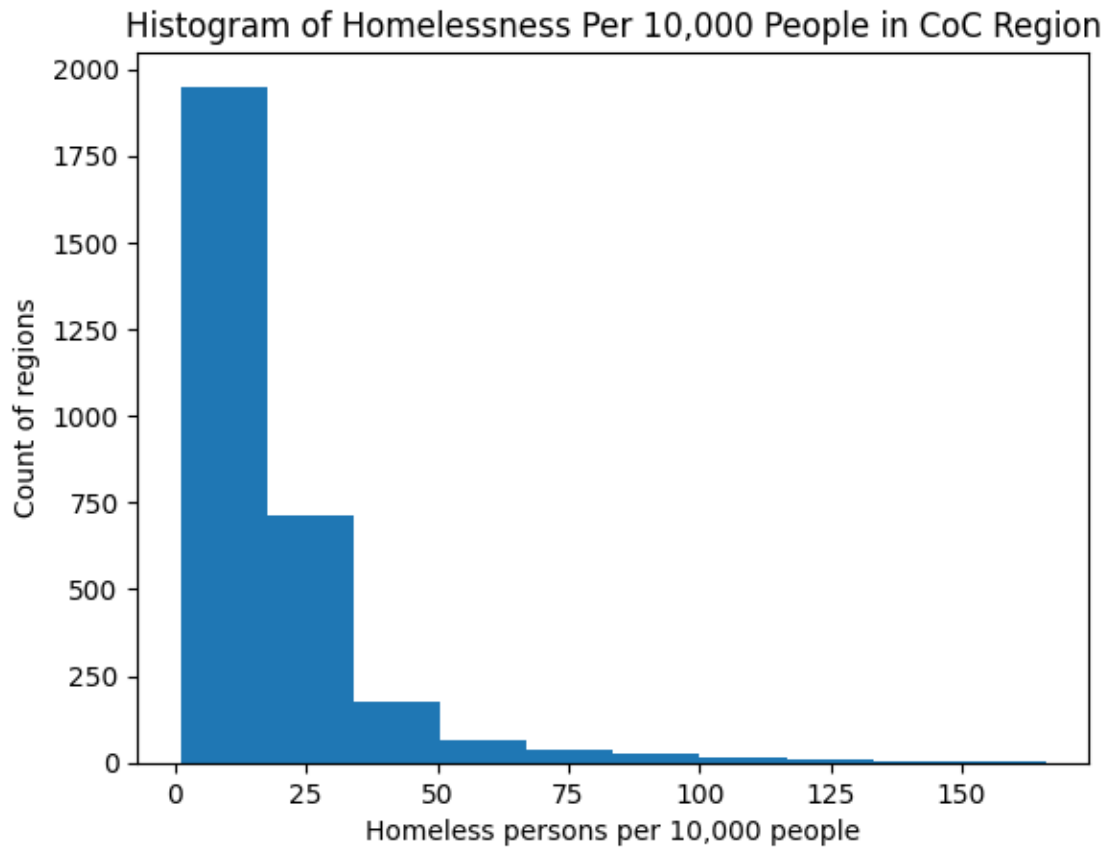
1.4 Visualizing the Data

For a preliminary data exploration, some basic visualizations will be provided:

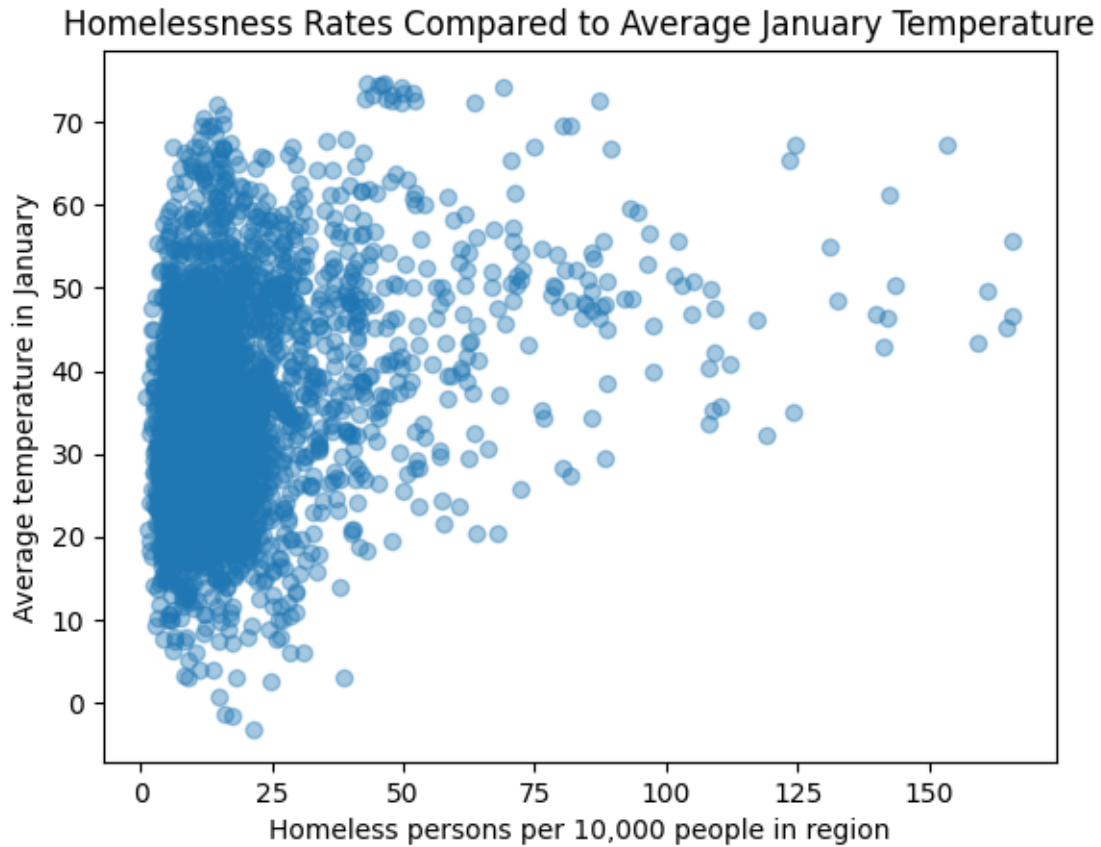
- Histogram of homelessness rates to see its distribution.
- Scatterplots of homelessness rates against selected features.

```
[23]: # Imports
import matplotlib.pyplot as plt
```

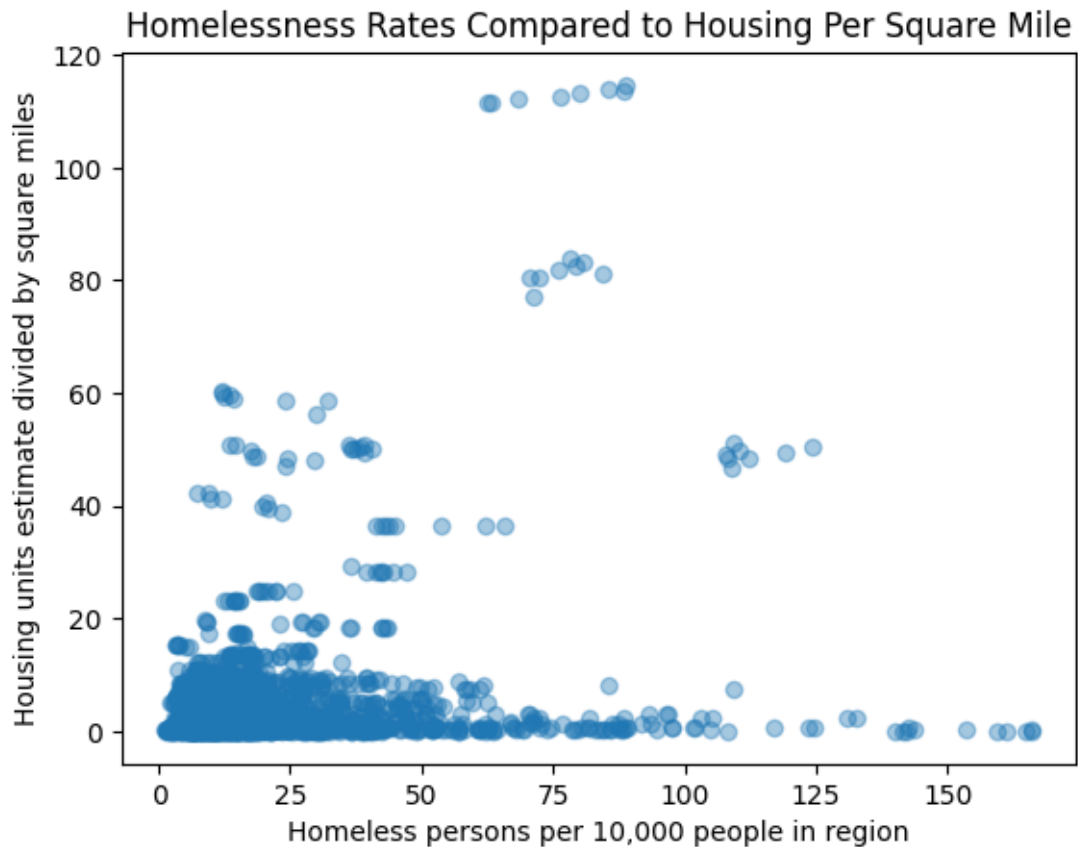
```
[24]: # Plot histo of homelessness rates using pit_hless_pit_hud_share
plt.hist(df_old['pit_hless_pit_hud_share'])
plt.title("Histogram of Homelessness Per 10,000 People in CoC Region")
plt.ylabel("Count of regions")
plt.xlabel("Homeless persons per 10,000 people")
plt.show()
```

```
[25]: # Plot homelessness rate against avg January temperature
plt.scatter(df_old['pit_hless_pit_hud_share'],
            df_old['env_wea_avgtemp_noaa'],
            alpha=0.4)
plt.title("Homelessness Rates Compared to Average January Temperature")
plt.xlabel("Homeless persons per 10,000 people in region")
plt.ylabel("Average temperature in January")
plt.show()
```

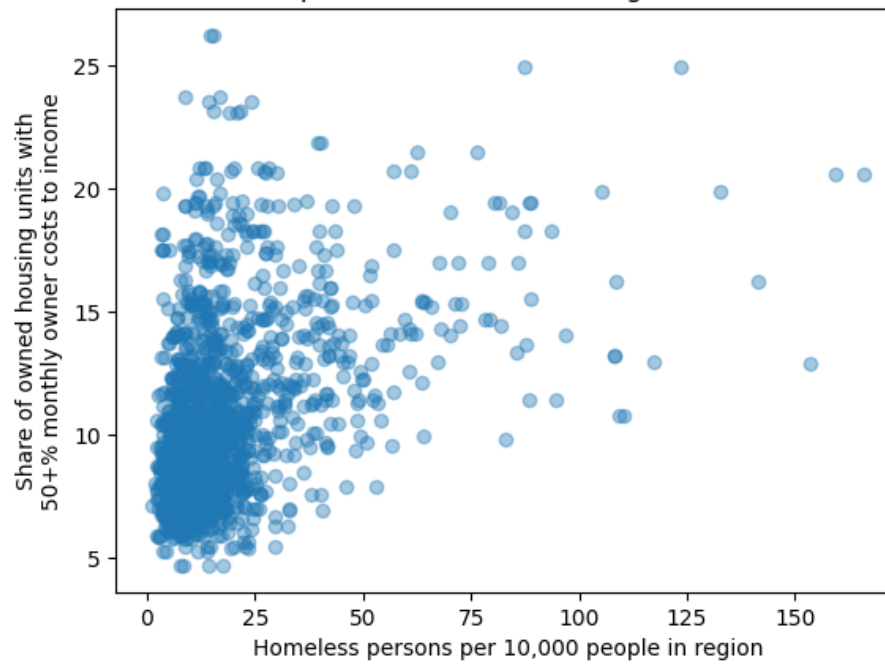


```
[26]: # Plot homelessness rate against housing units per square mile
plt.scatter(df_old['pit_hless_pit_hud_share'],
            df_old['hou_mkt_density_census'],
            alpha=0.4)
plt.title("Homelessness Rates Compared to Housing Per Square Mile")
plt.xlabel("Homeless persons per 10,000 people in region")
plt.ylabel("Housing units estimate divided by square miles")
plt.show()
```



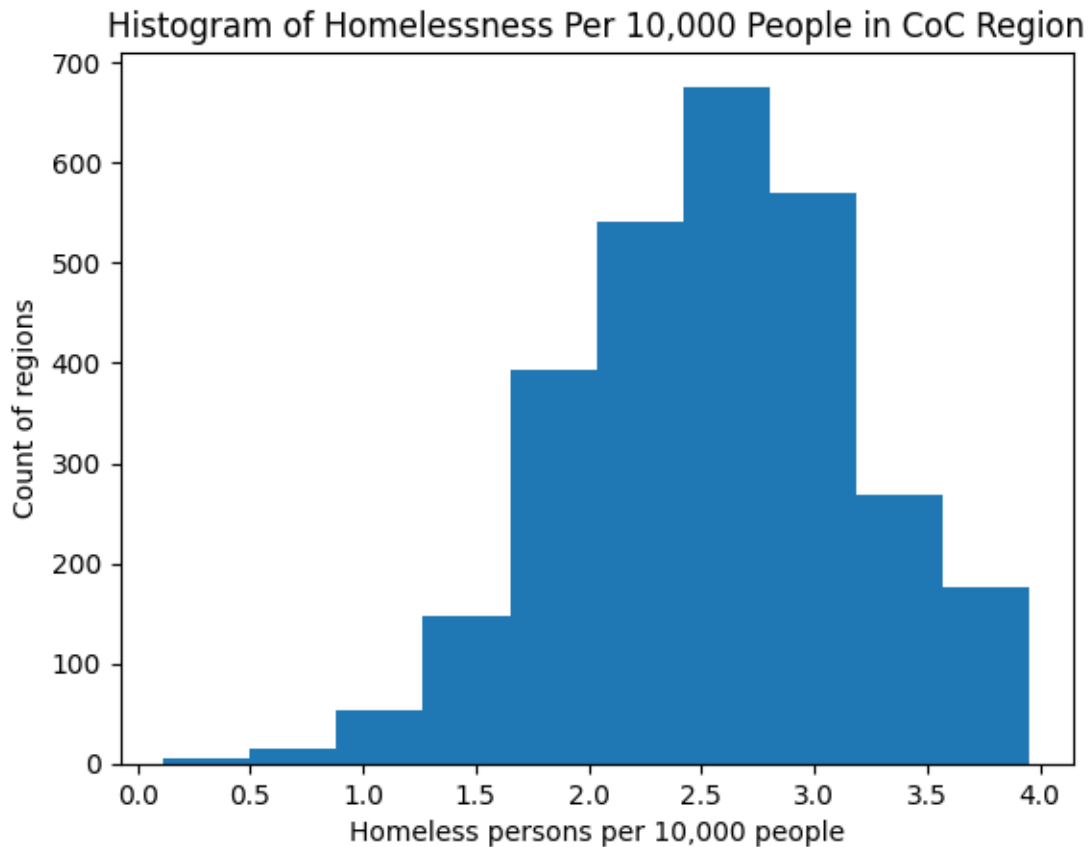
```
[27]: # Plot homelessness rate against severe housing burden rates
plt.scatter(df_old['pit_hless_pit_hud_share'],
            df_old['hou_mkt_burden_sev_own_xt'],
            alpha=0.4)
plt.title("Homelessness Rates Compared to Share of Housing Units With Severe_
↪Cost Burden")
plt.xlabel("Homeless persons per 10,000 people in region")
plt.ylabel("Share of owned housing units with\n50+% monthly owner costs to_
↪income")
plt.show()
```

Homelessness Rates Compared to Share of Housing Units With Severe Cost Burden

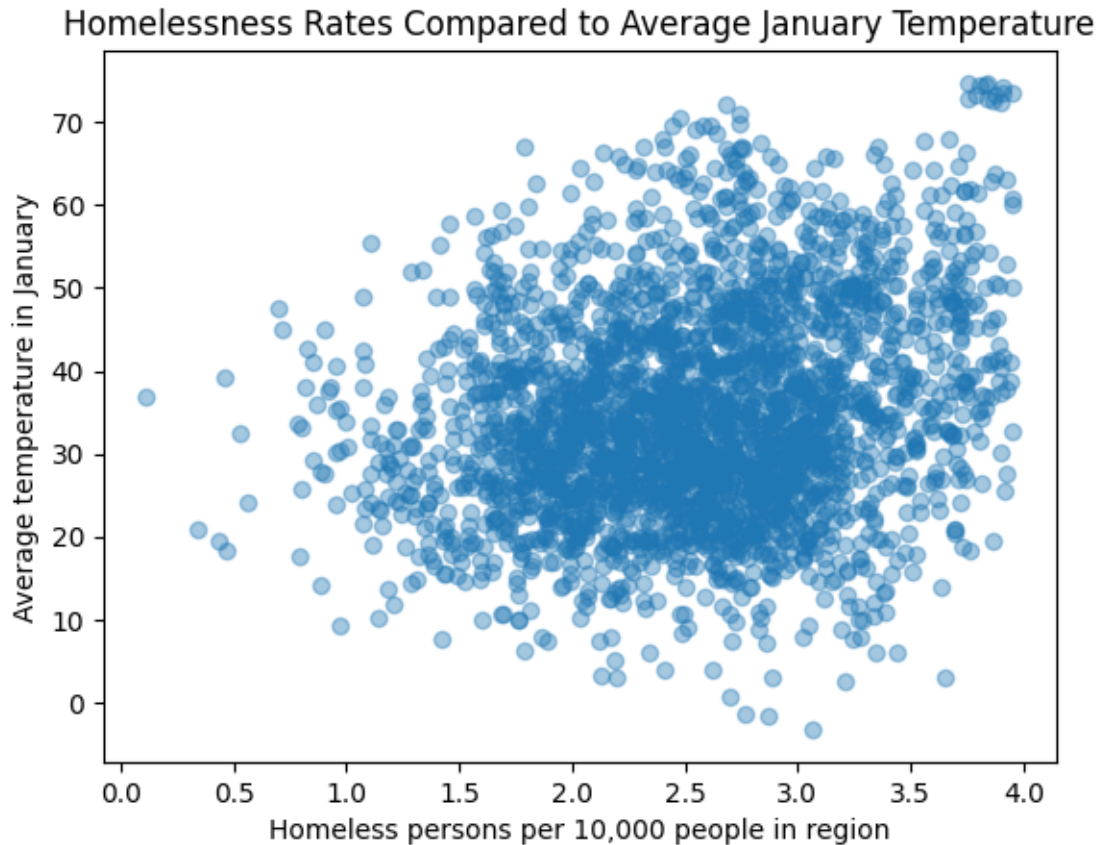


```
[28]: # Plot histo of log homelessness rates using pit_hless_pit_hud_share

plt.hist(df['log_hless'])
plt.title("Histogram of Homelessness Per 10,000 People in CoC Region")
plt.ylabel("Count of regions")
plt.xlabel("Homeless persons per 10,000 people")
plt.show()
```



```
[29]: # Plot homelessness rate against avg January temperature
plt.scatter(df['log_hless'],
            df['env_wea_avgtemp_noaa'],
            alpha=0.4)
plt.title("Homelessness Rates Compared to Average January Temperature")
plt.xlabel("Homeless persons per 10,000 people in region")
plt.ylabel("Average temperature in January")
plt.show()
```



2 Model Building and Evaluation

For this project I will build a multivariate linear regression model.

```
[30]: # Imports
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
```

```
[31]: # Train simple linear regression model
model = LinearRegression()
model.fit(X_train, y_train)
```

```
[31]: LinearRegression()
```

```
[32]: # Evaluate simple model
y_pred = model.predict(X_test)
```

```
rmse_1 = np.sqrt(mean_squared_error(y_test, y_pred))
r2_1 = r2_score(y_test, y_pred)
print(f"RMSE: {rmse_1}\nR2: {r2_1}")
```

RMSE: 0.5594410785154342

R2: 0.26496244138793423

```
[33]: # Compare RMSE against variables
np.max(df['log_hless'])
np.min(df['log_hless'])
print(f"Maximum log_hless: {np.max(df['log_hless'])}")
print(f"Mean log_hless: {np.mean(df['log_hless'])}")
print(f"Minimum log_hless: {np.min(df['log_hless'])}")
print(f"Range of log_hless: {np.max(df['log_hless']) - np.
    ↪min(df['log_hless'])}")
```

Maximum log_hless: 3.9533999972760756

Mean log_hless: 2.5473280605340425

Minimum log_hless: 0.11470397209702662

Range of log_hless: 3.838696025179049

The simple linear regression model has an R2 of 0.26 and an RMSE of 0.56. Ideally, both would be higher, but I would at least like for the RMSE to be higher.

To attempt improvement, I'll attempt a ridge regression model, which should be suitable because of the wide number of predictors. I'll also implement a StandardScaler, because scaling is important for the ridge approach. I'll also use GridSearchCV to tune hyperparameters.

```
[34]: # Ridge model param grid
param_grid = {
    'alpha': [0.1, 1.0, 10.0, 100.0],
    'solver': ['auto', 'svd', 'cholesky', 'lsqr', 'sparse_cg', 'sag', 'saga'],
    'max_iter': [15, 30, 50, 75, 100]
}
```

```
[35]: # Quiet ConvergenceWarnings found during gridsearch
import warnings
from sklearn.exceptions import ConvergenceWarning
warnings.filterwarnings("ignore", category=ConvergenceWarning)
```

```
[36]: # Grid search using ridge pipe
ridge_search = GridSearchCV(
    Ridge(random_state=0),
    param_grid,
    cv=5,
    scoring='neg_mean_squared_error',
    verbose=0
)
ridge_search.fit(X_train, y_train)
```

```
[36]: GridSearchCV(cv=5, estimator=Ridge(random_state=0),
                param_grid={'alpha': [0.1, 1.0, 10.0, 100.0],
                            'max_iter': [15, 30, 50, 75, 100],
                            'solver': ['auto', 'svd', 'cholesky', 'lsqr',
                                       'sparse_cg', 'sag', 'saga']},
                scoring='neg_mean_squared_error')
```

```
[37]: # Show best params
ridge_search.best_params_
```

```
[37]: {'alpha': 1.0, 'max_iter': 30, 'solver': 'sparse_cg'}
```

```
[38]: # Make scaler and ridge pipeline using best params
pipe = make_pipeline(
    StandardScaler(),
    Ridge(alpha=1.0, max_iter=30, solver='sparse_cg')
)
pipe.fit(X_train, y_train)
```

```
[38]: Pipeline(steps=[('standardscaler', StandardScaler()),
                      ('ridge', Ridge(max_iter=30, solver='sparse_cg'))])
```

```
[39]: y_pred_scaled = pipe.predict(X_test)
rmse_2 = np.sqrt(mean_squared_error(y_test, y_pred_scaled))
r2_2 = r2_score(y_test, y_pred_scaled)
print(f"RMSE: {rmse_2}\nR2: {r2_2}")
```

RMSE: 0.5594516068449951

R2: 0.26493477523765396

```
[40]: # Compare ridge vs standard model
print(f"Change in RMSE: {rmse_2 - rmse_1}")
print(f"Change in R^2: {r2_2 - r2_1}")
```

Change in RMSE: 1.0528329560832717e-05

Change in R²: -2.7666150280269974e-05

The performance change was negligible for the alternate approach, and the R2 did not change at all.

For further comparison, I will attempt a random forest approach as well:

```
[41]: # Imports
from sklearn.tree import DecisionTreeRegressor
```

```
[42]: # Train basic tree
tree = DecisionTreeRegressor(random_state=0)
tree_model = tree.fit(X_train, y_train)
```



```
[43]: # Evaluate
tree_preds = tree_model.predict(X_test)
rmse_3 = np.sqrt(mean_squared_error(y_test, tree_preds))
r2_3 = r2_score(y_test, tree_preds)
print(f"RMSE: {rmse_3}\nR2: {r2_3}")
```

RMSE: 0.4425683323055882

R2: 0.5399961186308797

This performance is vastly improved, with a significantly lower RMSE and a significantly higher R2. The tree approach is promising; however, trees are prone to overfitting, so I will test random forest instead:

```
[44]: # Imports
from sklearn.ensemble import RandomForestRegressor
```

```
[45]: # Train random forest regressor
forest = RandomForestRegressor(random_state=0)
forest_model = forest.fit(X_train, y_train)
```

```
[46]: # Evaluate
forest_preds = forest_model.predict(X_test)
rmse_4 = np.sqrt(mean_squared_error(y_test, forest_preds))
r2_4 = r2_score(y_test, forest_preds)
print(f"RMSE: {rmse_4}\nR2: {r2_4}")
```

RMSE: 0.29798430791150254

R2: 0.7914608424119127

```
[47]: # Find feature importance of random forest predictors
importances = forest_model.feature_importances_
indices = np.argsort(importances)[::-1]
names = [X_train.columns[i] for i in indices]
```

```
[48]: # Plot feature importance
plt.bar(range(X_train.shape[1]), importances[indices])
plt.xticks(range(X_train.shape[1]), names, rotation=90)
plt.title("Feature importances in random forest regressor model")
plt.xlabel("Feature name")
plt.ylabel("Importance value")
plt.show()
```

